

## Chapter 26

### ECC Accelerator (ECC)

#### 26.1 Overview

Elliptic Curve Cryptography (ECC) is an approach to public-key cryptography based on the algebraic structure of elliptic curves. ECC uses smaller keys compared to RSA cryptography while providing equivalent security.

ESP32-P4's ECC accelerator can complete various calculations based on different elliptic curves, thus accelerating the ECC algorithm and ECC-derived algorithms such as ECDSA.

#### 26.2 Feature List

ESP32-P4's ECC accelerator has the following features:

- Three different elliptic curves, namely, P-192, P-256, and P-384 defined in [FIPS 186-5](#)
- 11 working modes
- Enhanced anti-attack performance
- Interrupt upon completion of calculation

#### 26.3 ECC Basics

To better illustrate the functionality of the ECC accelerator, the basic knowledge and terminology used in this chapter are introduced in this section.

##### 26.3.1 Elliptic Curve and Points on the Curves

The ECC algorithm is based on elliptic curves over prime fields, which can be represented as:

$$y^2 = x^3 + ax + b \bmod p$$

where,

- $p$  is a prime number,
- $a$  and  $b$  are two non-negative integers smaller than  $p$ ,
- and  $(x, y)$  is a point on the curve satisfying the representation.

## 26.3.2 Affine Coordinates and Jacobian Coordinates

An elliptic curve can be represented as below:

- In affine coordinates:

$$y^2 = x^3 + ax + b \bmod p$$

- In Jacobian coordinates:

$$Y^2 = X^3 + aXZ^4 + bZ^6 \bmod p$$

To convert affine coordinates  $(x, y)$  to/from Jacobian coordinates  $(X, Y, Z)$ :

- From Jacobian to Affine coordinates:

$$x = X/Z^2 \bmod p$$

$$y = Y/Z^3 \bmod p$$

- From Affine to Jacobian coordinates:

$$X = x$$

$$Y = y$$

$$Z = 1$$

## 26.3.3 Memory Blocks

ECC's memory blocks store the input and output data of the ECC operation.

Table 26.3-1. ECC Accelerator Memory Blocks

| Memory Block    | Size (byte) | Starting Address * | Ending Address * | Access |
|-----------------|-------------|--------------------|------------------|--------|
| ECC_MULT_Mem_k  | 32          | 0x100              | 0x11F            | R/W    |
| ECC_MULT_Mem_Px | 32          | 0x120              | 0x13F            | R/W    |
| ECC_MULT_Mem_Py | 32          | 0x140              | 0x15F            | R/W    |
| ECC_MULT_Mem_Qx | 32          | 0x160              | 0x17F            | R/W    |
| ECC_MULT_Mem_Qy | 32          | 0x180              | 0x19F            | R/W    |
| ECC_MULT_Mem_Qz | 32          | 0x1A0              | 0x1BF            | R/W    |

\* Address offset related to the ECC accelerator base address is provided in Table 7.3-2 in Chapter 7 [System and Memory](#).

## 26.3.4 Data and Data Block

ESP32-P4's ECC can operate on 192-bit, 256-bit, or 384-bit data, depending on the elliptic curves.

For example, the 256-bit data  $D[255 : 0]$  can be divided into eight 32-bit data blocks

$D[n][31 : 0] (n = 0, 1, \dots, 7)$ . Data blocks with smaller indexes correspond to lower binary bits. To be specific:

$$D[255 : 0] = D[7][31 : 0], D[6][31 : 0], D[5][31 : 0], D[4][31 : 0], D[3][31 : 0], D[2][31 : 0], D[1][31 : 0], D[0][31 : 0]$$

### 26.3.5 Writing Data

Writing data means writing data to an ECC memory block and using this data as the input to the ECC algorithm. To be specific, writing data to an ECC memory block means writing  $D[n][31:0]$  ( $n = 0, 1, \dots, 7$ ) to the “starting address of this ECC memory block +  $4 \times n$ ” sequentially:

- write  $D[0]$  to “starting address”
- write  $D[1]$  to “starting address + 4”
- ...
- write  $D[7]$  to “starting address + 28”

**Note:**

When storing data, write only the data block of the required length. Do not append a 0 to the most significant bit. For example, for 192-bit data, store only the 192 bits without appending 0.

### 26.3.6 Reading Data

Read data means reading data from the starting address of an ECC memory block and using this data as the output from the ECC algorithm. To be specific, read data from an ECC memory block means reading  $D[n][31:0]$  ( $n = 0, 1, \dots, 7$ ) from the “starting address of this ECC memory block +  $4 \times n$ ” successively:

- read  $D[0]$  from “starting address”
- read  $D[1]$  from “starting address + 4”
- ...
- read  $D[7]$  from “starting address + 28”

**Note:**

When reading data, only the data block of the required length needs to be read. For example, to read 192-bit data, simply read the lower 192 bits (i.e., 6 data blocks).

### 26.3.7 Standard Calculation and Jacobian Calculation

ESP32-P4's ECC performs Affine Point Calculation (including Affine Point Verification, Affine Point Add, and Affine Point Multiplication) using the affine coordinates and Jacobian Calculation (including Jacobian Point Verification, Jacobian Point Add, and Jacobian Point Multiplication) using the Jacobian coordinates.

## 26.4 Function Description

### 26.4.1 Curve Mode

ESP32-P4's ECC supports acceleration based on three curve modes, each corresponding to an elliptic curve. By configuring the `ECC_MULT_CURVE_MODE` field, users can select the desired key size. For details, see Table 26.4-1 below.

Table 26.4-1. ECC Accelerator Curve Mode Selection

| ECC_MULT_CURVE_MODE | Elliptic Curves* |
|---------------------|------------------|
| 0                   | FIPS P-192       |
| 1                   | FIPS P-256       |
| 2                   | FIPS P-384       |

\* See definition of FIPS P-192, P-256, and P-384 defined in [FIPS 186-5](#).

## 26.4.2 Working Modes

ESP32-P4's ECC accelerator supports 11 working modes based on two elliptic curves described in the above section. By configuring the `ECC_MULT_WORK_MODE` field, users can select the desired working mode. For details, see Table 26.4-2.

Table 26.4-2. Working Modes of ECC Accelerator

| ECC_MULT_WORK_MODE | Working Modes                             |
|--------------------|-------------------------------------------|
| 0                  | Affine Point Multi                        |
| 1                  | <b>Reserved</b>                           |
| 2                  | Affine Point Verif                        |
| 3                  | Affine Point Verif + Multi                |
| 4                  | Jacobian Point Multi                      |
| 5                  | Point Add                                 |
| 6                  | Jacobian Point Verif                      |
| 7                  | Affine Point Verif + Jacobian Point Multi |
| 8                  | Mod Add                                   |
| 9                  | Mod Sub                                   |
| 10                 | Mod Multi                                 |
| 11                 | Mod Div                                   |

**Note:**

Note that the calculation of Jacobian Point Multi mode is about 10% faster than that of the Affine Point Multi mode.

Detailed descriptions about different working modes are provided in the following sections.

### 26.4.2.1 Affine Point Multiplication (Affine Point Multi)

Affine Point Multiplication can be represented as:

$$Q = (Q_x, Q_y) = (J_x, J_y, J_z) = k \cdot (P_x, P_y)$$

where,

- $(Q_x, Q_y)$  is the affine expression of point Q.
- $(J_x, J_y, J_z)$  is the Jacobian expression of point Q.

- Input:  $P_x$ ,  $P_y$ , and  $k$  are stored in [ECC\\_MULT\\_Mem\\_Px](#), [ECC\\_MULT\\_Mem\\_Py](#), and [ECC\\_MULT\\_Mem\\_k](#) respectively.
- Output:  $Q_x$  and  $Q_y$  are stored in [ECC\\_MULT\\_Mem\\_Px](#) and [ECC\\_MULT\\_Mem\\_Py](#) respectively.

### 26.4.2.2 Affine Point Verification (Affine Point Verif)

Affine Point Verification can be used to verify if a point  $(P_x, P_y)$  is on a selected elliptic curve.

- Input:  $P_x$  and  $P_y$  are stored in [ECC\\_MULT\\_Mem\\_Px](#) and [ECC\\_MULT\\_Mem\\_Py](#) respectively.
- Output: The verification result is stored in the [ECC\\_MULT\\_VERIFICATION\\_RESULT](#) bit.

### 26.4.2.3 Affine Point Verification + Affine Point Multiplication (Affine Point Verif + Multi)

In this mode, ECC first verifies if point  $(P_x, P_y)$  is on the selected elliptic curve. If so, the following multiplication is performed:

$$Q = (Q_x, Q_y) = (J_x, J_y, J_z) = k \cdot (P_x, P_y)$$

where,

- $(Q_x, Q_y)$  is the affine expression of point Q.
- $(J_x, J_y, J_z)$  is the Jacobian expression of point Q.
- Input:  $P_x$ ,  $P_y$ , and  $k$  are stored in [ECC\\_MULT\\_Mem\\_Px](#), [ECC\\_MULT\\_Mem\\_Py](#), and [ECC\\_MULT\\_Mem\\_k](#) respectively.
- Output:
  - The verification result is stored in the [ECC\\_MULT\\_VERIFICATION\\_RESULT](#) bit.
  - $Q_x$  and  $Q_y$  are stored in [ECC\\_MULT\\_Mem\\_Px](#) and [ECC\\_MULT\\_Mem\\_Py](#) respectively.
  - $J_x$ ,  $J_y$ , and  $J_z$  are stored in [ECC\\_MULT\\_Mem\\_Qx](#), [ECC\\_MULT\\_Mem\\_Qy](#), and [ECC\\_MULT\\_Mem\\_Qz](#).

### 26.4.2.4 Jacobian Point Multiplication (Jacobian Point Multi)

Jacobian Point Multiplication can be represented as:

$$Q = (Q_x, Q_y, Q_z) = k \cdot (P_x, P_y, 1)$$

where,

- $(Q_x, Q_y, Q_z)$  is the Jacobian expression of point Q.
- 1 in the point's Jacobian coordinates is automatically completed by hardware.
- Input:  $P_x$ ,  $P_y$ , and  $k$  are stored in [ECC\\_MULT\\_Mem\\_Px](#), [ECC\\_MULT\\_Mem\\_Py](#), and [ECC\\_MULT\\_Mem\\_k](#) respectively.
- Output:  $Q_x$ ,  $Q_y$ , and  $Q_z$  are stored in [ECC\\_MULT\\_Mem\\_Qx](#), [ECC\\_MULT\\_Mem\\_Qy](#), and [ECC\\_MULT\\_Mem\\_Qz](#) respectively.

### 26.4.2.5 Point Addition (Point Add)

Point Addition can be represented as:

$$R = (R_x, R_y) = (J_x, J_y, J_z) = (P_x, P_y, 1) + (Q_x, Q_y, Q_z)$$

where,

- $(R_x, R_y)$  is the affine expression of point R.
- $(J_x, J_y, J_z)$  is the Jacobian expression of point R.
- 1 in the point's Jacobian coordinates is automatically completed by hardware.
- Input:
  - $P_x$  and  $P_y$  are stored in [ECC\\_MULT\\_Mem\\_Px](#) and [ECC\\_MULT\\_Mem\\_Py](#).
  - $Q_x$ ,  $Q_y$ , and  $Q_z$  are stored in [ECC\\_MULT\\_Mem\\_Qx](#), [ECC\\_MULT\\_Mem\\_Qy](#), and [ECC\\_MULT\\_Mem\\_Qz](#).
- Output:
  - $R_x$  and  $R_y$  are stored in [ECC\\_MULT\\_Mem\\_Px](#) and [ECC\\_MULT\\_Mem\\_Py](#).
  - $J_x$ ,  $J_y$  and  $J_z$  are stored in [ECC\\_MULT\\_Mem\\_Qx](#), [ECC\\_MULT\\_Mem\\_Qy](#), and [ECC\\_MULT\\_Mem\\_Qz](#).

### 26.4.2.6 Jacobian Point Verification (Jacobian Point Verif)

Jacobian Point Verification can be used to verify if point  $(Q_x, Q_y, Q_z)$  is on a selected elliptic curve.

- $(Q_x, Q_y, Q_z)$  is the Jacobian expression of point Q.
- Input:  $Q_x$ ,  $Q_y$ , and  $Q_z$  are stored in [ECC\\_MULT\\_Mem\\_Qx](#), [ECC\\_MULT\\_Mem\\_Qy](#), and [ECC\\_MULT\\_Mem\\_Qz](#) respectively.
- Output: The verification result is stored in the [ECC\\_MULT\\_VERIFICATION\\_RESULT](#) bit.

### 26.4.2.7 Affine Point Verification + Jacobian Point Multiplication (Affine Point Verif + Jacobian Point Multi)

In this mode, ECC first verifies if point  $(P_x, P_y)$  is on the selected elliptic curve. If so, the following multiplication is performed:

$$Q = (Q_x, Q_y, Q_z) = k \cdot (P_x, P_y, 1)$$

where,

- $(Q_x, Q_y, Q_z)$  is the Jacobian expression of point Q.
- 1 in the point's Jacobian coordinates is automatically completed by hardware.
- Input:  $P_x$ ,  $P_y$ , and  $k$  are stored in [ECC\\_MULT\\_Mem\\_Px](#), [ECC\\_MULT\\_Mem\\_Py](#), and [ECC\\_MULT\\_Mem\\_k](#).
- Output:
  - The verification result is stored in the [ECC\\_MULT\\_VERIFICATION\\_RESULT](#) bit.
  - $Q_x$ ,  $Q_y$ , and  $Q_z$  are stored in [ECC\\_MULT\\_Mem\\_Qx](#), [ECC\\_MULT\\_Mem\\_Qy](#), and [ECC\\_MULT\\_Mem\\_Qz](#).

### 26.4.2.8 Mod Addition (Mod Add)

Mod Addition can be represented as:

$$R = A + B \bmod N$$

where,

- Input:
  - A and B are stored in [ECC\\_MULT\\_Mem\\_Px](#) and [ECC\\_MULT\\_Mem\\_Py](#).
  - The value of N is related to the register fields below:
    - \* [ECC\\_MULT\\_CURVE\\_MODE](#) to select the related curve.
    - \* [ECC\\_MULT\\_MOD\\_BASE](#) to choose using mod base or order of the base point.
- Output: R is stored in [ECC\\_MULT\\_Mem\\_Px](#).

### 26.4.2.9 Mod Subtraction (Mod Sub)

Mod Subtraction can be represented as:

$$R = A - B \bmod N$$

where,

- Input:
  - A and B are stored in [ECC\\_MULT\\_Mem\\_Px](#) and [ECC\\_MULT\\_Mem\\_Py](#).
  - The value of N is related to the register fields below:
    - \* [ECC\\_MULT\\_CURVE\\_MODE](#) to select the related curve.
    - \* [ECC\\_MULT\\_MOD\\_BASE](#) to choose using mod base or order of the base point.
- Output: R is stored in [ECC\\_MULT\\_Mem\\_Px](#).

### 26.4.2.10 Mod Multiplication (Mod Multi)

Mod Multiplication can be represented as:

$$R = A \cdot B \bmod N$$

where,

- Input:
  - A and B are stored in [ECC\\_MULT\\_Mem\\_Px](#) and [ECC\\_MULT\\_Mem\\_Py](#).
  - The value of N is related to the register fields below:
    - \* [ECC\\_MULT\\_CURVE\\_MODE](#) to select the related curve.
    - \* [ECC\\_MULT\\_MOD\\_BASE](#) to choose using mod base or order of the base point.
- Output: R is stored in [ECC\\_MULT\\_Mem\\_Py](#).

### 26.4.2.11 Mod Division (Mod Div)

Mod Division can be represented as:

$$R = A \cdot B^{-1} \bmod N$$

where,

- Input:
  - A and B are stored in [ECC\\_MULT\\_Mem\\_Px](#) and [ECC\\_MULT\\_Mem\\_Py](#).
  - The value of N is related to the register fields below:
    - \* [ECC\\_MULT\\_CURVE\\_MODE](#) to select the related curve.
    - \* [ECC\\_MULT\\_MOD\\_BASE](#) to choose using mod base or order of the base point.
- Output: R is stored in [ECC\\_MULT\\_Mem\\_Py](#).

## 26.4.3 Enhancing Anti-Attack Performance

When performing different types of point multiplication calculations, i.e., mode 0, 3, 4, and 7, ESP32-P4's ECC accelerator offers an additional option to further enhance its anti-attack performance by ensuring that all point multiplication calculations take the same maximum runtime.

To enable the enhanced anti-attack performance, set [ECC\\_MULT\\_SECURITY\\_MODE](#) to 1. If this setting is enabled, each time the ECC accelerator perform a point multiplication calculation:

- The execution latency is constant for a given operation.
- Power consumption variations are minimized.

## 26.5 Clock and Reset

ESP32-P4's ECC only has one clock module (CRYPTO\_ECC\_CLK) and one reset module (CRYPTO\_ECC\_RST).

ECC's clock and reset is handled by the the Power/Clock/Reset (PCR) module (see Chapter 10 [Reset and Clock](#) for more information). Users should enable the ECC clock by setting [HP\\_SYS\\_CLKRST\\_CRYPTOECC\\_CLK\\_EN](#) and release the ECC reset by clearing [HP\\_SYS\\_CLKRST\\_RST\\_EN\\_ECC](#) before starting the ECC accelerator. Besides, due to resource reuse between cryptography accelerator modules, users also need to additionally clear the [HP\\_SYS\\_CLKRST\\_RST\\_EN\\_ECDSA](#) bit.

## 26.6 Interrupts

ESP32-P4's ECC accelerator can generate the ECC\_INTR interrupt signal that will be sent to the [Interrupt Matrix](#).

ECC\_INTR has only one interrupt source to generate the ECC\_INTR interrupt signal, i.e., [ECC\\_MULT\\_CALC\\_DONE\\_INT](#), which is triggered on the completion of an ECC calculation.



Each interrupt source can be configured by a common set of registers that are described in Section [Interrupt Configuration Registers](#). The ECC\_MULT\_CALC\_DONE\_INT interrupt source is configured by the following registers (refer to Section [26.8 Register Summary](#) for more information):

- [ECC\\_MULT\\_CALC\\_DONE\\_INT\\_RAW](#): Stores the raw interrupt status of ECC\_MULT\_CALC\_DONE\_INT.
- [ECC\\_MULT\\_CALC\\_DONE\\_INT\\_ST](#): Indicates the status of the ECC\_MULT\_CALC\_DONE\_INT interrupt. This bit is generated by enabling or disabling the [ECC\\_MULT\\_CALC\\_DONE\\_INT\\_RAW](#) bit via [ECC\\_MULT\\_CALC\\_DONE\\_INT\\_ENA](#).
- [ECC\\_MULT\\_CALC\\_DONE\\_INT\\_ENA](#): Enables or disables the ECC\_MULT\_CALC\_DONE\_INT interrupt.
- [ECC\\_MULT\\_CALC\\_DONE\\_INT\\_CLR](#): Set this bit to clear the ECC\_MULT\_CALC\_DONE\_INT interrupt status. By setting this bit to 1, the [ECC\\_MULT\\_CALC\\_DONE\\_INT\\_RAW](#) and [ECC\\_MULT\\_CALC\\_DONE\\_INT\\_ST](#) bits will be cleared.

**Note:**

For definitions of *interrupt*, *interrupt signal*, *interrupt source*, and their correlations, please refer to Chapter [12 Interrupt Matrix](#) > Section [12.2 Interrupt Terminology in ESP32-P4](#).

## 26.7 Programming Procedures

The programming procedure for configuring ECC is described below:

1. Configure the ECC clock and reset. Refer to Section [26.5](#) for detailed information.
2. Select the key size and working mode as described in Section [26.4](#).
3. Enable the [ECC\\_MULT\\_CALC\\_DONE\\_INT](#) interrupt as described in Section [26.6](#).
4. Set the [ECC\\_MULT\\_START](#) field to start ECC calculation.
5. Wait for the [ECC\\_MULT\\_CALC\\_DONE\\_INT](#) interrupt, which indicates the completion of the ECC calculation.
6. Check the result as described in Section [26.4](#).

## 26.8 Register Summary

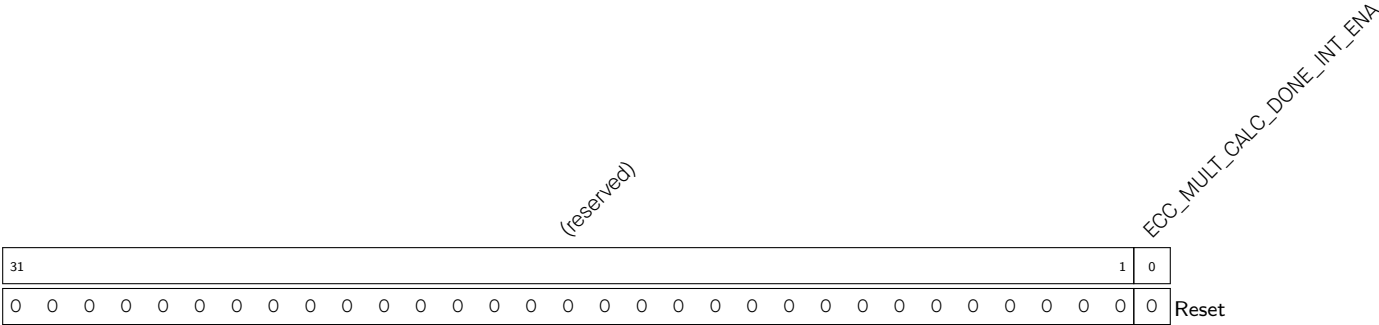
The addresses in this section are relative to ECC Accelerator base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

| Name                                 | Description                          | Address | Access   |
|--------------------------------------|--------------------------------------|---------|----------|
| <b>Interrupt Registers</b>           |                                      |         |          |
| <a href="#">ECC_MULT_INT_RAW_REG</a> | ECC raw interrupt status register    | 0x000C  | R/SS/WTC |
| <a href="#">ECC_MULT_INT_ST_REG</a>  | ECC masked interrupt status register | 0x0010  | RO       |
| <a href="#">ECC_MULT_INT_ENA_REG</a> | ECC interrupt enable register        | 0x0014  | R/W      |
| <a href="#">ECC_MULT_INT_CLR_REG</a> | ECC interrupt clear register         | 0x0018  | WT       |
| <b>Configuration Register</b>        |                                      |         |          |
| <a href="#">ECC_MULT_CONF_REG</a>    | ECC configuration register           | 0x001C  | varies   |
| <b>Version Register</b>              |                                      |         |          |
| <a href="#">ECC_MULT_DATE_REG</a>    | Version control register             | 0x00FC  | R/W      |

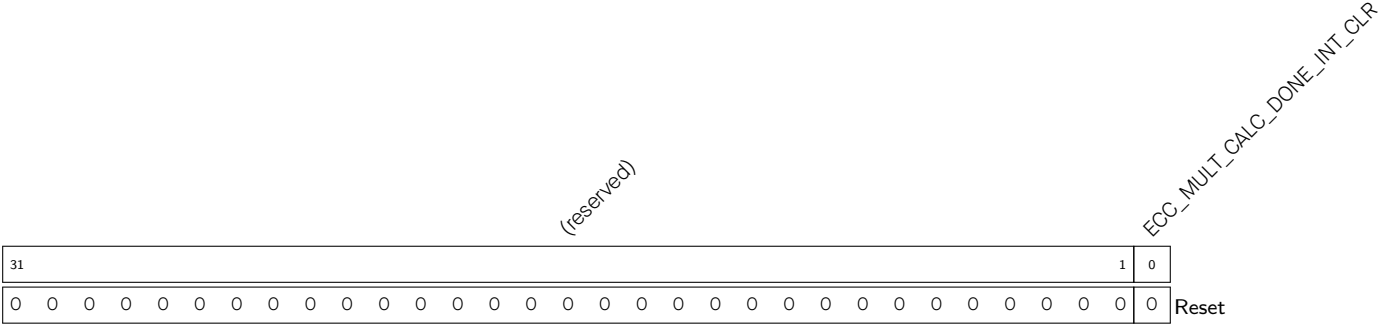


Register 26.3. ECC\_MULT\_INT\_ENA\_REG (0x0014)



ECC\_MULT\_CALC\_DONE\_INT\_ENA Write 1 to enable the ECC\_MULT\_CALC\_DONE\_INT interrupt.  
(R/W)

Register 26.4. ECC\_MULT\_INT\_CLR\_REG (0x0018)



ECC\_MULT\_CALC\_DONE\_INT\_CLR Write 1 to clear the ECC\_MULT\_CALC\_DONE\_INT interrupt. (WT)

### Register 26.5. ECC\_MULT\_CONF\_REG (0x001C)

Diagram illustrating the ECC\_MULT\_SECURITY\_MODE register structure (32 bits):

- Bit 31: ECC\_MULT\_MEM\_CLOCK\_GATE\_FORCE\_ON
- Bit 30: ECC\_MULT\_CLK\_EN
- Bit 29: ECC\_MULT\_VERIFICATION\_RESULT
- Bits 10-9: (reserved)
- Bit 8: ECC\_MULT\_SECURITY\_MODE
- Bits 5-4: ECC\_MULT\_WORK\_MODE
- Bits 3-2: ECC\_MULT\_MOD\_BASE
- Bit 1: ECC\_MULT\_CURVE\_MODE
- Bit 0: ECC\_MULT\_RESET

Reset

**ECC\_MULT\_START** Configures whether to start the calculation of the ECC accelerator. This bit will be self-cleared when the calculation is done.

0: No effect

1: Start the calculation of the ECC accelerator

(R/W/SC)

**ECC\_MULT\_RESET** Configures whether to reset the ECC accelerator.

0: No effect

1: Reset

(WT)

**ECC\_MULT\_CURVE\_MODE** Configures the curve mode of the ECC accelerator.

0: P-192

1: P-256

2: P-384

3: Invalid

(R/W)

**ECC\_MULT\_MOD\_BASE** Configures whether to choose using the mod base or order of base point in the mod operation. Only valid in working modes 8-11.

0:  $n$  (order of base point)

1:  $p \pmod{\text{base of curve}}$

(R/W)

**ECC\_MULT\_WORK\_MODE** Configures the working mode of the ECC accelerator.

0: Affine Point Multi mode

1: Reserved

## 2: Affine Point Verif mode

### 3: Affine Point Verif + Multi mode

#### 4: Jacobian Point Multi mode

## 5: Point Add mode

## 6: Jacobian Point Verif mode

## 7: Affine Point Verif + Jacobian Point Multi mode

8: Mod Add mode

## 9. Mod Sub mode

## 10: Mod Multi mode

11: Mod Div mode

Register 26.5. ECC\_MULT\_CONF\_REG (0x001C)

Continued from the previous page...

**ECC\_MULT\_SECURITY\_MODE** Configures whether to enhance the anti-attack performance when calculating point multiplication. Only valid in working modes 0, 3, 4, and 7.  
0: Disable  
1: Enable  
(R/W)

**ECC\_MULT\_VERIFICATION\_RESULT** Represents the verification result of the ECC accelerator, valid only when the calculation is done.  
0: Verification failed  
1: Verification passed  
(R/SS)

**ECC\_MULT\_CLK\_EN** Configures whether to force on register clock gate.  
0: No effect  
1: Force on  
(R/W)

**ECC\_MULT\_MEM\_CLOCK\_GATE\_FORCE\_ON** Configures whether to force on ECC memory clock gate.  
0: No effect  
1: Force on  
(R/W)

Register 26.6. ECC\_MULT\_DATE\_REG (0x00FC)

|            |    |    |   |               |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |       |
|------------|----|----|---|---------------|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|-------|
| (reserved) |    |    |   | ECC_MULT_DATE |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |       |
| 31         | 28 | 27 |   |               |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | 0     |
| 0          | 0  | 0  | 0 | 0x2408120     |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  | Reset |

**ECC\_MULT\_DATE** Version control register. (R/W)